

Step 3 — Scaling to 3+ Tools With a Hard Step Limit and Failure Recovery — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — 3 tools, a raw counter, no real recovery message

```
# tools.py
from langchain_core.tools import tool
from pydantic import BaseModel

class CalculatorArgs(BaseModel):
    expression: str

@tool(args_schema=CalculatorArgs)
def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression."""
    return str(eval(expression))

class WeatherArgs(BaseModel):
    city: str

@tool(args_schema=WeatherArgs)
def lookup_weather(city: str) -> str:
    """Look up the current weather for a city."""
    return f"It is sunny and 72F in {city}." # stands in for Doc02's real HTTP

class OrderArgs(BaseModel):
    order_id: str

@tool(args_schema=OrderArgs)
def flaky_lookup(order_id: str) -> str:
    """Look up an order's status."""
    if order_id == "BAD-ORDER":
```

```
raise RuntimeError("order service timed out")
return f"Order {order_id} shipped yesterday."
```

```
# agent.py
class MaxIterationsExceeded(Exception):
    pass

def run_agent(task, max_iterations=6):
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator, lookup_weather])
    messages = [SystemMessage("Use the right tool for each task."), HumanMessage(task)]
    tools_by_name = {"calculator": calculator, "lookup_weather": lookup_weather,
                    "lookup_facts": lookup_facts}
    round_count = 0

    while True:
        round_count += 1
        if round_count > max_iterations:
            raise MaxIterationsExceeded("too many rounds")

        response = model.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return response.content

        for call in response.tool_calls:
            try:
                result = tools_by_name[call["name"]].invoke(call["args"])
            except Exception as e:
                result = f"Error: {e}"
            messages.append(ToolMessage(content=result, tool_call_id=call["id"]))
```

Expected output, run against "What's the status of order BAD-ORDER?" :

```
The order lookup failed (order service timed out). I wasn't able to retrieve the
```

This works — 3 tools, a limit, a failure that doesn't crash the loop. `MaxIterationsExceeded("too many rounds")` carries no information about *which* task or what actually happened before it hit the limit, and the tool descriptions ("Look up the current weather for a city.", "Look up an order's status.") are generic enough that a model choosing between `lookup_weather` and a hypothetical "look up a city's facts" tool would have nothing to disambiguate on.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — structured `AgentResult`, typed, logged per round

```
# agent.py
from pydantic import BaseModel
from langchain_core.messages import SystemMessage, HumanMessage, ToolMessage
from langchain_openai import ChatOpenAI
from logging_setup import get_logger
from tools import calculator, lookup_weather, flaky_lookup

logger = get_logger(__name__)

class AgentStep(BaseModel):
    tool: str
    args: dict
    result: str
    ok: bool

class AgentResult(BaseModel):
    answer: str
    steps: list[AgentStep]

class MaxIterationsExceeded(Exception):
    def __init__(self, task: str, steps: list[AgentStep]):
        super().__init__(f"Hit max_iterations on task: {task!r} after {len(steps)} steps")
        self.task = task
        self.steps = steps

TOOLS = [calculator, lookup_weather, flaky_lookup]
TOOLS_BY_NAME = {t.name: t for t in TOOLS}

def run_agent(task: str, max_iterations: int = 6) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools(TOOLS)
    messages = [SystemMessage("Use the right tool for each task. Never guess a number"),
                HumanMessage(task)]
```

```

steps: list[AgentStep] = []
round_count = 0

while True:
    round_count += 1
    if round_count > max_iterations:
        raise MaxIterationsExceeded(task, steps)

    logger.info("Round %d: asking model", round_count)
    response = model.invoke(messages)
    messages.append(response)

    if not response.tool_calls:
        return AgentResult(answer=response.content, steps=steps)

    for call in response.tool_calls:
        tool_fn = TOOLS_BY_NAME[call["name"]]
        try:
            result = tool_fn.invoke(call["args"])
            ok = True
        except Exception as e:
            result = f"Error: {e}"
            ok = False
        logger.info(" %s(%s) -> %s [ok=%s]", call["name"], call["args"], result, ok)
        steps.append(AgentStep(tool=call["name"], args=call["args"], result=result, ok=ok))
        messages.append(ToolMessage(content=result, tool_call_id=call["id"]))

```

```

# main.py
from agent import run_agent, MaxIterationsExceeded

try:
    result = run_agent("What's the status of order BAD-ORDER?")
    print(result.answer)
except MaxIterationsExceeded as e:
    print(f"Failed: {e}")
    for step in e.steps:
        print(f" {step.tool}({step.args}) -> {step.result} [ok={step.ok}]")

```

Expected output:

```
The order lookup for BAD-ORDER failed (order service timed out). I wasn't able to
```

Difference from Basic: `MaxIterationsExceeded` still carries no context in this version's raise site — that's what Advanced fixes. What's different here is everything else: `AgentStep` / `AgentResult` give the run real structure, `TOOLS_BY_NAME` replaces the inline dict built fresh each call, and every

round is logged through Doc01's `logger` instead of only being visible if something happens to print it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — descriptions that actually disambiguate, and a limit error you can debug from

```
# tools.py (descriptions sharpened)
@tool(args_schema=WeatherArgs)
def lookup_weather(city: str) -> str:
    """Look up the CURRENT WEATHER for a named city (temperature, conditions).
    Use this ONLY for weather. Do NOT use this for general facts about a place,
    like its population, capital, or history – you already know those."""
    return f"It is sunny and 72F in {city}."

@tool(args_schema=OrderArgs)
def flaky_lookup(order_id: str) -> str:
    """Look up a SPECIFIC ORDER's shipping status by its order ID.
    Use this ONLY when the user names or implies a specific order.
    Do NOT use this for general questions about shipping policy or return windows
    if order_id == "BAD-ORDER":
        raise RuntimeError("order service timed out")
    return f"Order {order_id} shipped yesterday."
```

```
# agent.py (the limit check moves before the model call, and the exception carries
def run_agent(task: str, max_iterations: int = 6) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools(TOOLS)
    messages = [SystemMessage("Use the right tool for each task. Never guess a number"),
                 HumanMessage(task)]
    steps: list[AgentStep] = []
    round_count = 0

    while True:
        round_count += 1
        if round_count > max_iterations:
            logger.error("Hit max_iterations=%d on task: %r", max_iterations, task)
            raise MaxIterationsExceeded(task, steps) # never even makes the model call

        response = model.invoke(messages)
        messages.append(response)
```

```

if not response.tool_calls:
    return AgentResult(answer=response.content, steps=steps)

for call in response.tool_calls:
    tool_fn = TOOLS_BY_NAME[call["name"]]
    try:
        result = tool_fn.invoke(call["args"])
        ok = True
    except Exception as e:
        result = f"Error: {e}"
        ok = False
    steps.append(AgentStep(tool=call["name"], args=call["args"], result=result, ok=ok))
    messages.append(ToolMessage(content=result, tool_call_id=call["id"]))

```

Expected output, run against a prompt engineered to loop (like "Keep checking order BAD-ORDER's status until you get a real answer" with `max_iterations=3`):

```

Failed: Hit max_iterations on task: 'Keep checking order BAD-ORDER's status until
flaky_lookup({'order_id': 'BAD-ORDER'}) -> Error: order service timed out [ok=F
flaky_lookup({'order_id': 'BAD-ORDER'}) -> Error: order service timed out [ok=F
flaky_lookup({'order_id': 'BAD-ORDER'}) -> Error: order service timed out [ok=F

```

Whoever catches `MaxIterationsExceeded` can now see exactly what task was running and exactly what every attempted tool call returned — no need to reproduce the failure to understand it.

Approach 2 — an unclear-prompt test that actually proves the descriptions work

```

# test_agent.py (excerpt)
def test_ambiguous_prompt_picks_weather_not_order():
    # "conditions" could plausibly mean either "weather conditions" or "order con
    # a model reading vague tool descriptions could go either way
    result = run_agent("What are the current conditions in Paris?")
    tools_called = [step.tool for step in result.steps]
    assert tools_called == ["lookup_weather"]

```

Expected output: the test passes because `lookup_weather`'s description explicitly claims "weather" and explicitly excludes general facts, while `flaky_lookup`'s description explicitly scopes itself to a named order — neither description leaves room for "conditions in Paris" to plausibly mean an order lookup.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's tool descriptions were plain summaries and its exception carried no context — both technically worked, neither would survive the README's own test cases well. Approach 1 fixes the exception (full context, logged, and the limit is checked before wasting a model call on a round that can't complete)

and sharpens the descriptions with explicit "use this / not this" language. Approach 2 doesn't change the agent at all — it's the concrete evidence that the description-sharpening in Approach 1 actually did something, by testing a prompt built to be ambiguous on purpose.

Which one should you actually write? All of Approach 1 — a debuggable

`MaxIterationsExceeded` and disambiguating tool descriptions are both cheap to write and expensive to be missing the one time a real user hits either case. Approach 2's specific test is worth keeping in `test_agent.py` as written — it's exactly the kind of test that silently starts failing the moment someone adds a fourth tool with an overlapping job, which is precisely when you'd want to know.